

ENGINEERING SPRINT 10

Component Communication, Forms, LDS and Reusable LWC Architecture

Student Placement Portal – Sprint Implementation and Architecture Report

This report presents the Sprint 10 implementation through the concepts covered in Chapter 10: focused LWC components, parent-child communication, custom events, forms, validation, Lightning Data Service, reactive data, refresh behaviour, reusable components, and consistent UI states.

1. Sprint Overview

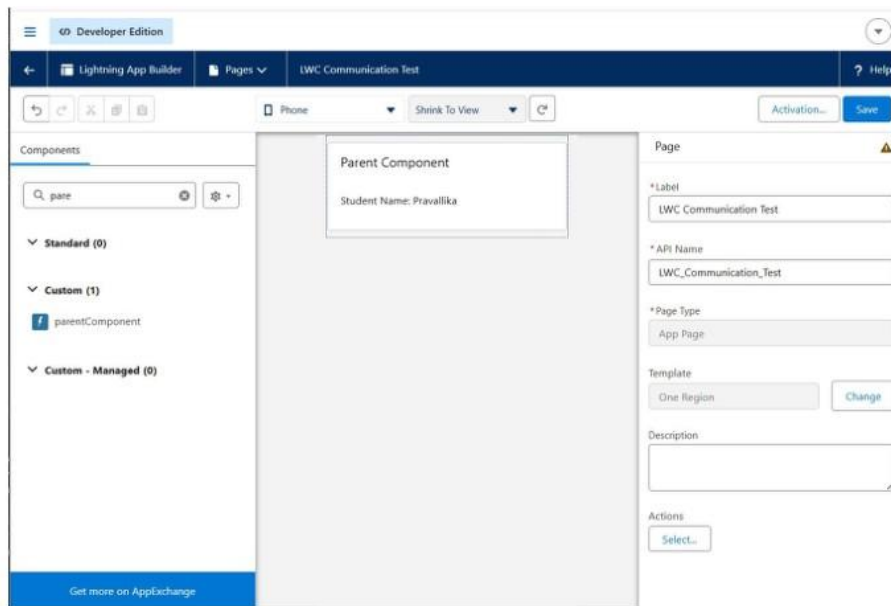
A component becomes truly useful when it knows not only how to perform its own task, but also how to participate in a larger system. Sprint 10 extends the Student Placement Portal from individual screens into a coordinated application.

The chapter defines the main learning outcomes as designing groups of LWCs as one application, understanding parent-to-child and child-to-parent communication, using public properties and custom events appropriately, building forms with Lightning base components, separating client-side and server-side validation, using Lightning Data Service where appropriate, understanding reactive data and refresh behaviour, designing reusable components, handling loading, success, empty and error states, and identifying unnecessary coupling.

Student Placement Portal Component Family

```
StudentPortal
■
■■■ StudentSummary
■■■ StudentProfile
■■■ EligibleJobs
■ ■■■ JobCard
■■■ MyApplications
■ ■■■ ApplicationCard
■■■ OfferSummary
```

Each component should have a clear responsibility and a clear relationship with other components. Focused components communicate to form one coherent application.



2. Parent-to-Child Communication

Parent-to-child communication is used when a parent component owns information that a child component needs. In LWC, the parent can pass information to a child through a public property exposed with `@api`.

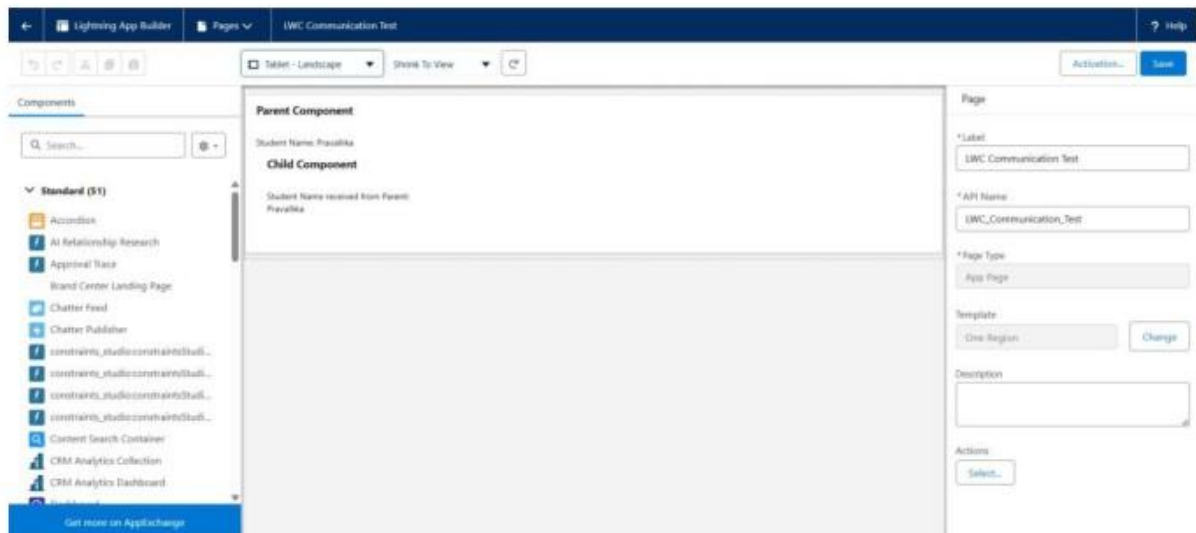
```
// Child
@api job;

// Parent
<c-job-details job={selectedJob}></c-job-details>
```

The parent should deliberately pass the information the child needs. This avoids unnecessary duplicate retrieval and makes data ownership easier to understand.

Engineering Principle

Give a component what it needs. If the parent already has the required information, the child can receive it instead of independently retrieving the same data without a clear reason.



3. Child-to-Parent Communication

A child component communicates an action or occurrence to its parent through a custom event. The child should not directly manipulate the parent's state.

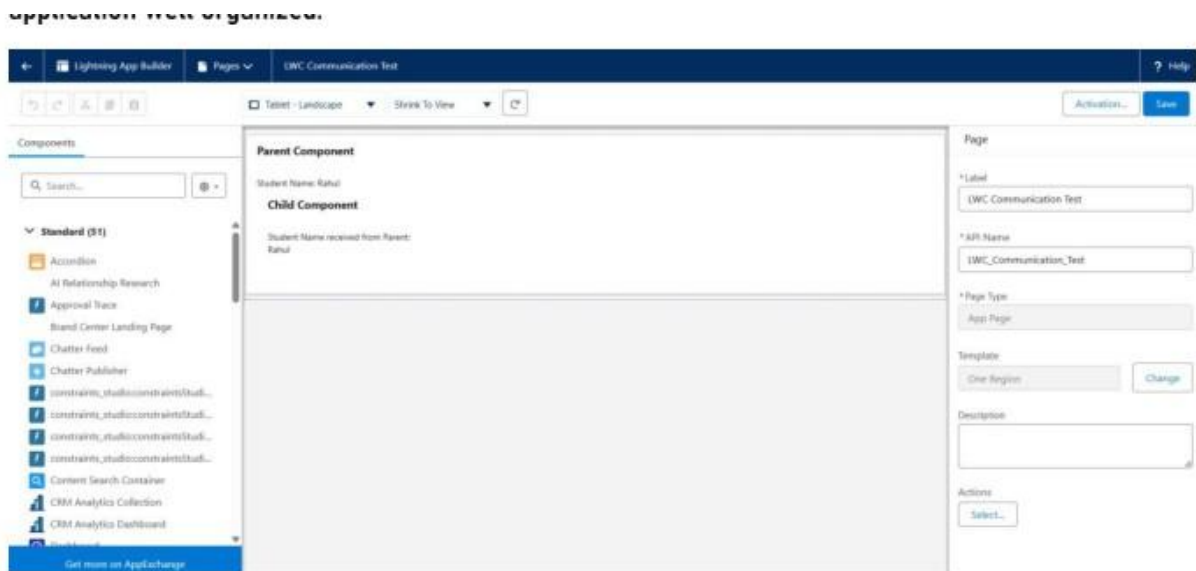
```
this.dispatchEvent( new
CustomEvent('viewdetails', {
detail: { jobId: this.job.Id }
})
);
```

The parent listens for the event and decides what should happen next. This keeps the child focused on its own responsibility while allowing the parent to coordinate application behaviour.

Communication Flow

```
Child
↓
Custom Event
↓
Parent
↓
Parent decides what to do
```

For sibling components, communication should normally pass through their common parent: one child reports an event, the parent handles it, and the parent passes the required information to the other child.



4. Custom Events and Event Contracts

The JobCard can report meaningful actions such as viewdetails, apply, or favorite. The event should contain only the information the parent actually needs.

```
this.dispatchEvent( new
CustomEvent('apply', { detail:
{ jobId: this.job.Id } })
);
```

Intent vs Outcome

An event should communicate a fact that the child actually knows. A user clicking Apply expresses intent; it does not by itself prove that an application was successfully created. The business outcome is determined after the appropriate processing succeeds.

Children report events. Parents coordinate behaviour. Events should communicate facts rather than unsupported assumptions. Clear event contracts reduce tight coupling between components.

5. Lightning Base Components

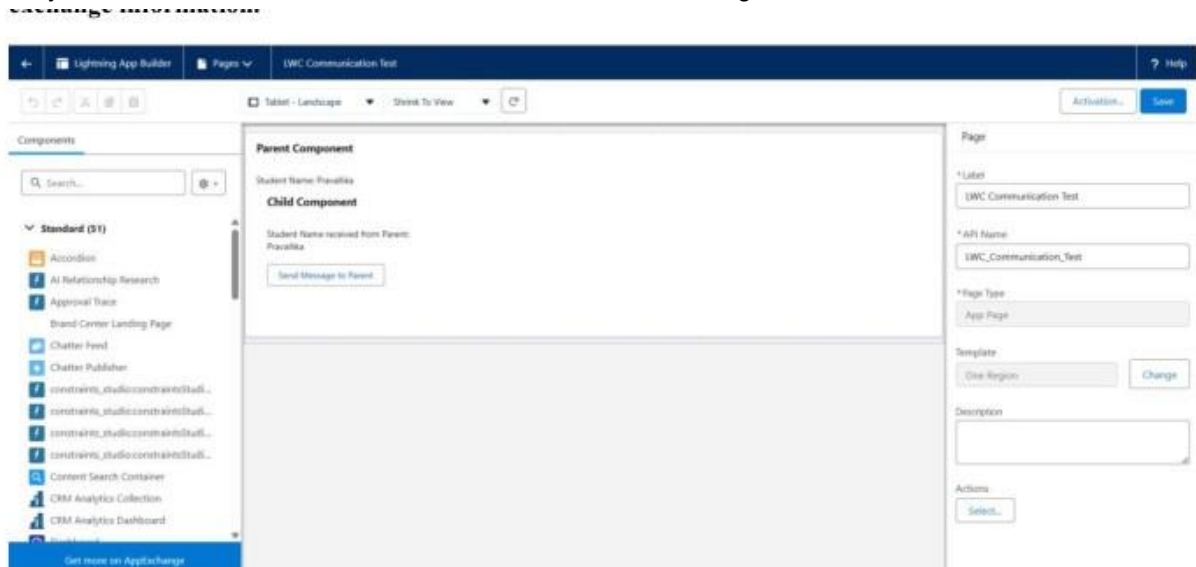
Salesforce provides Lightning base components for common interface requirements. Reusing these components provides standard behaviour and styling consistent with Salesforce Lightning experiences.

Examples include lightning-button, lightning-card, lightning-input, lightning-combobox, lightning-textarea, lightning-checkbox-group and lightning-radio-group.

Example Form Field

```
<lightning-input  
  label="Phone" value={phone}  
  onchange={handlePhoneChange}>  
</lightning-input>
```

The engineering principle is to reuse the platform before reinventing the platform. Abstraction should still match the complexity of the actual form; a small form does not need an elaborate generic framework.



6. Student Profile Form

The Student Profile component is responsible for loading current information, displaying it, allowing editing, validating required fields, saving changes, showing success or meaningful errors, and refreshing displayed information appropriately.

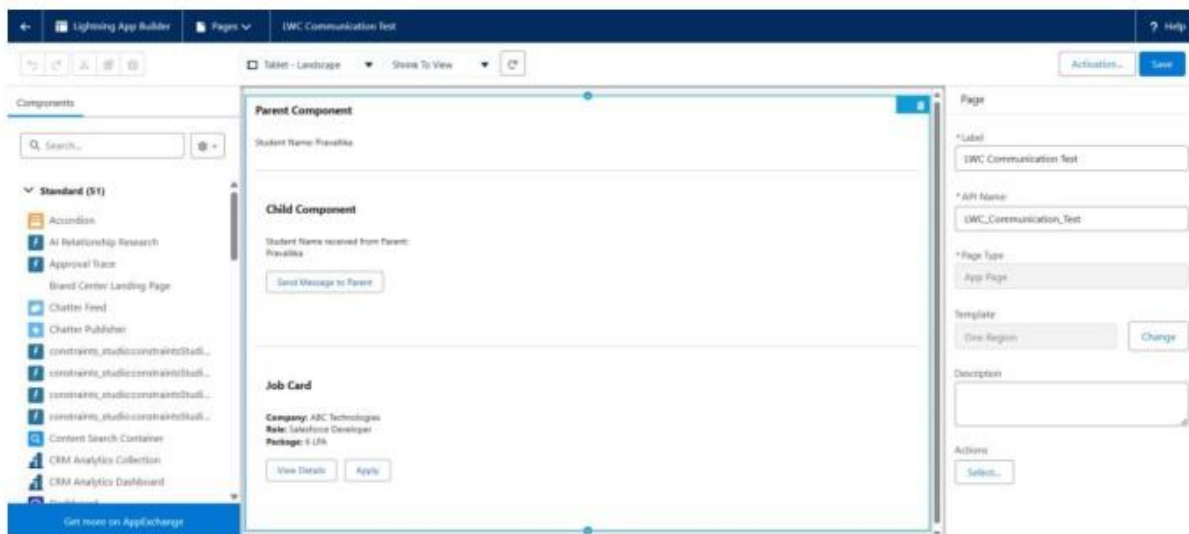
Fields include Phone, Email, Branch, CGPA, Skills and Preferred Location.

The uploaded implementation uses the Student_Profile__c object for the student profile data. The profile form therefore acts as the user-facing layer for maintaining that Salesforce record.

Validation

Required-field validation ensures that mandatory information is present before saving. CGPA is treated as a numeric field and the implementation restricts it to the range 0 to 10.

the data. This reduces duplicate code and keeps the application organized.



7. Client-Side and Server-Side Validation

Client-side validation and server-side validation have different responsibilities. Client-side validation gives immediate feedback and improves the user experience. Server-side validation remains authoritative because a client can be bypassed.

Client Validation
↓
Better User Experience

Server Validation
↓
Business Integrity

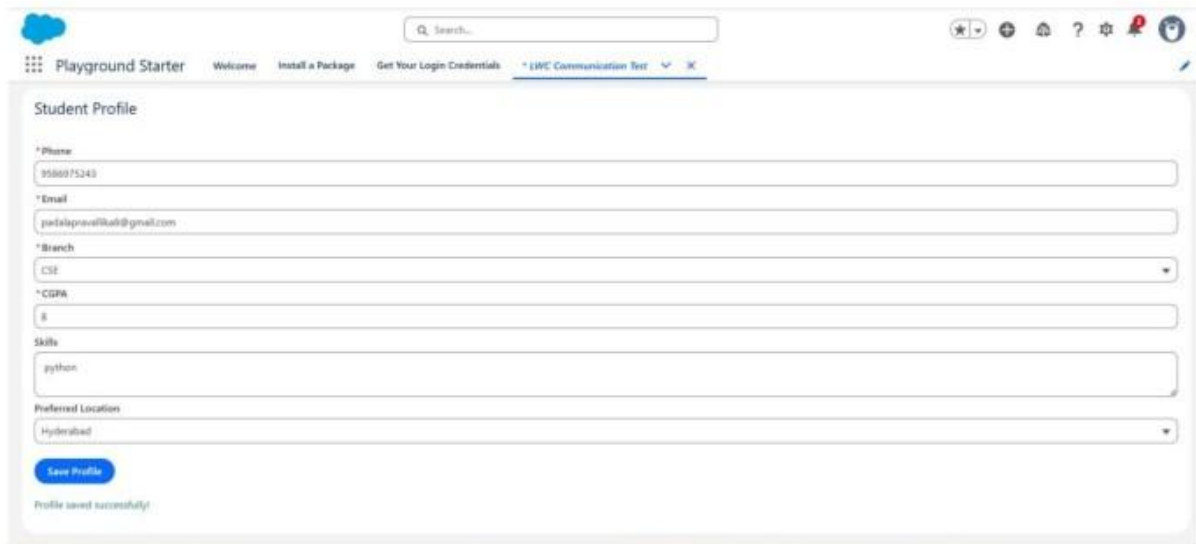
For the Student Profile, required fields are checked before submission and CGPA is restricted to a valid range. The implementation also uses backend Apex for the Save Profile and Get Student Profile operations.

Save Profile Flow

Enter Details
↓
Validate Fields
↓
Validate CGPA
↓
Send to Apex
↓
Save Record
↓

Show Success Message

The component provides an event to the parent component for managing student details.



The screenshot shows a web application interface for a 'Student Profile'. The page has a header with a search bar and navigation links: 'Playground Starter', 'Welcome', 'Install a Package', 'Get Your Login Credentials', and 'LWC Communication Test'. The main form is titled 'Student Profile' and contains several input fields: 'Phone' (with value '9580975243'), 'Email' (with value 'joshalpravalikar@gmail.com'), 'Branch' (a dropdown menu with 'CSE' selected), 'CGPA' (with value '8'), 'Skills' (a text area with 'python'), and 'Preferred Location' (a dropdown menu with 'Hyderabad' selected). A blue 'Save Profile' button is at the bottom left. Below the button, a green message states 'Profile saved successfully!'.

8. Apex Integration and Load Profile

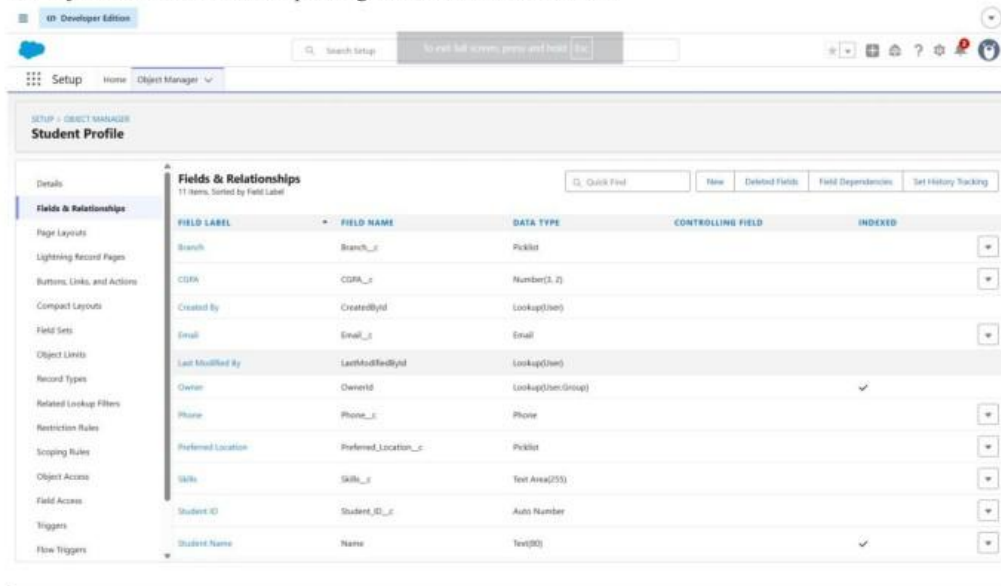
In the uploaded implementation, Apex handles the two main Student Profile operations: saving student details in Salesforce and retrieving an existing student profile.

```
Student Profile LWC
↓
Apex Controller
↓
Salesforce Object
```

The Load Profile functionality uses the student's email address to identify the profile. The retrieved information is then displayed in the Student Profile form.

```
Enter Email
↓
Click Load Profile
↓
Apex searches for the profile
↓
Profile information is retrieved
↓
Fields are populated
```

The object contains fields corresponding to the Student Profile form.



FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Branch	Branch__c	Picklist		
CDRA	CDRA__c	Number(3, 2)		
Created By	CreatedById	Lookup(User)		
Email	Email__c	Email		
Last Modified By	LastModifiedById	Lookup(User)		
Owner	OwnerId	Lookup(User/Group)		✓
Phone	Phone__c	Phone		
Preferred Location	PreferredLocation__c	Picklist		
Skills	Skills__c	Text Area(255)		
Student ID	Student_ID__c	Auto Number		
Student Name	Name	Text(50)		✓

9. Lightning Data Service and Architecture Decision

Lightning Data Service provides standard Salesforce mechanisms for working with records and can reduce the need for custom Apex in common record operations.

The chapter asks the team to choose deliberately between LDS-based record operations, custom Apex, or a combination. The requirement determines the architecture; Apex should not be selected merely because the team already knows Apex.

Decision in This Sprint

For this uploaded implementation, Apex was used for the custom Save Profile and Load Profile functions. LDS remains an important platform capability to consider when a supported record operation can be handled without custom server-side logic.

The screenshot shows the 'Student Profile' section of the AWS Playground Starter interface. At the top, there is a navigation bar with the AWS logo, the text 'Playground Starter', and several links: 'Welcome', 'Install a Package', 'Get Your Login Credentials', and two tabs for 'IWC Customization Test'. A search bar is located in the top right corner. Below the navigation bar, the 'Student Profile' section contains several form fields, each with a red border and a red 'X' icon in the top left corner, indicating they are required and currently empty. The fields are: 'Phone', 'Email', 'Branch' (a dropdown menu), 'CGPA', 'Skills' (a text area), 'Preferred Location' (a dropdown menu), and a 'Save Profile' button at the bottom left.

A change to the Student record can affect eligible jobs, application eligibility, and the student summary. This creates a network of dependent information.

Example

If StudentSummary shows CGPA 8.1 while EligibleJobs uses CGPA 7.9, the problem is not merely a UI bug. It is an architectural data-ownership problem.

The screenshot shows a web application interface. At the top, there is a navigation bar with a search bar and several icons. Below the navigation bar, there is a tabbed interface with tabs labeled 'Playground Starter', 'Welcome', '* LWC Communication Test', and '* More'. The 'Student Profile' form is displayed below the tabs. It contains several input fields with validation errors:

- * Phone: 5484 (Error: Please enter a valid 10-digit phone number.)
- * Email: njdnkkd (Error: Enter a valid email address, such as name@email.com.)
- * Branch: IT (No error)
- * CGPA: 12 (Error: The number is too high.)
- Skills: (Empty field)
- Preferred Location: Select an Option (Dropdown menu)

11. Refreshing After Save

After a profile update, dependent components may contain stale information. For example, if CGPA changes from 7.4 to

8.2, the Student Summary may update while Eligible Jobs still shows the old eligibility.

The data can be correct in Salesforce while the screen is stale. The architecture must identify what changed, which component owns the data, which components depend on it, and how those components should refresh.

Practical Strategies

Parent-owned state; custom events; refreshing appropriate wired data; LDS-supported record notifications and reactive updates; and re-querying data when genuinely necessary.

The simplest communication architecture that keeps the UI consistent should be preferred.

Profile Loaded Successfully

Student Profile

*Phone
8521647399

*Email
patsalp@gmail.com

*Branch
CSE

*CGPA
8.00

Skills
java

Preferred Location
Bangalore

Load Profile Save Profile

Profile loaded successfully!

12. Loading, Success, Empty and Error States

A polished application communicates its state so that the user never has to guess whether an operation is still running, completed successfully, has no data, or failed.

Loading – Loading your profile... | Editing – Normal form. | Saving – Saving... | Success – Profile updated successfully. | Error – We could not update your profile. Please review the highlighted fields.

The uploaded implementation handles Loading, Success and Error states for Save and Load operations. It displays success messages such as Profile saved successfully! and Profile loaded successfully!, and shows an error message when communication with Apex fails.



13. Reusable Components

Reusable components can be used in multiple places without creating a new component each time. In the Student Placement Portal, a JobCard can display different job records using different data, reducing duplicate code and improving maintenance.

StatusBadge Example

```
ApplicationCard
```

```
↓
```

```
StatusBadge
```

```
InterviewCard
```

```
↓
```

```
StatusBadge
```

```
OfferCard
```

```
↓
```

```
StatusBadge
```

A reusable component should provide a meaningful capability. Good names communicate intent, such as `statusBadge`, `applicationStatus`, `jobCard`, `emptyState` and `loadingIndicator`.

Reuse vs Over-Engineering

Reuse becomes counterproductive when abstraction adds more complexity than the duplication it removes. The goal is meaningful reuse, not creating a component for every small piece of markup.

This provides clear feedback to the user.

The screenshot shows a web application interface for a 'Student Profile'. The top navigation bar includes links like 'Playground Starter', 'Welcome', 'Install a Package', 'Get Your Login Credentials', and 'LWC Communication Test'. The main content area is titled 'Student Profile' and contains several input fields: 'Phone' (9588752872), 'Email' (joshajpravalika@gmail.com), 'Branch' (ECE), 'CGPA' (8.00), 'Skills' (python), and 'Preferred Location' (Bangalore). Below the form are 'Load Profile' and 'Save Profile' buttons. A green box highlights the 'Save Profile' button, and a message 'Profile loaded successfully!' is displayed at the bottom of the form area.

14. Avoiding the 'God Component'

A large component that handles job search, job details, applications, withdrawals, interviews, offers, profiles, notifications and admin actions becomes difficult to understand and maintain.

A healthier architecture distributes responsibilities among focused components while allowing them to communicate explicitly.

```
StudentPortal
├── StudentSummary
├── StudentProfile
├── EligibleJobs
├── JobCard
├── MyApplications
├── ApplicationCard
└── OfferSummary
```

The parent can coordinate, while children remain focused on their individual responsibilities. This reduces unnecessary coupling and makes the system easier to explain and evolve.

Search...

Playground StarterWelcomeLWC Communication TestMore

Student Profile

* Phone

5484

Please enter a valid 10-digit phone number.

* Email

njdnnkd

Enter a valid email address, such as name@email.com.

* Branch

IT

* CGPA

12

The number is too high.

Skills

Preferred Location

Select an Option

This shows how different components work together as one complete Salesforce application.

Search...

Playground StarterWelcomeInstall a PackageGet Your Login CredentialsLWC Communication TestPravallika | Student Profile

Student Profile

Pravallika

New ContactEditNew Opportunity

RelatedDetails

Student Name

Pravallika

Phone

9560264135

Email

pravallika@gmail.com

Branch

CSE

CGPA

9.50

Skills

python, java

Preferred Location

Hyderabad

Created By

PADALA DURGA SEJHA PRAVALLIKA - 10/06/2026, 4:19 pm

Owner

PADALA DURGA SEJHA PRAVALLIKA

Last Modified By

PADALA DURGA SEJHA PRAVALLIKA - 10/06/2026, 4:19 pm

